

Inleiding

Deze documentatie beschrijft de implementatie van de geautomatiseerde cloud orkestratie (opdracht 3), waarbij gebruik wordt gemaakt van Terraform voor infrastructuurprovisioning en Ansible voor Kubernetes cluster configuratie. De oplossing is ontwikkeld om te voldoen aan de eisen van een multi cloud architectuur met focus op vendor onafhankelijkheid.

Leeruitkomsten

3.1 Provision a set of resources using Terraform in a multi-cloud environment

Status: Behaald

De oplossing gebruikt Terraform voor het volledig geautomatiseerd provisioneren van AWS infrastructuur:

- **VPC en Networking:** Volledige netwerkconfiguratie met publieke/private subnets, NAT gateways en een internet gateway
- **EKS Cluster:** Managed Kubernetes cluster met worker nodes verspreid over meerdere availability zones
- **RDS Database:** Managed PostgreSQL database voor applicatie data storage
- **ECR Repository:** Container registry voor het hosten van Docker images
- **Security Groups:** Netwerk beveiligingsregels voor alle componenten

3.2 Deploy a microservices solution on Kubernetes

Status: Behaald

De CloudShirt .NET webapplicatie wordt gedeployed als microservice architectuur:

- **Containerization:** Applicatie is gecontaineriseerd via Docker en gepusht naar AWS ECR
- **Kubernetes Deployment:** 5 replica pods voor high availability en werk verdeling
- **Health Checks:** Check naar readiness en liveness voor robuuste service delivery
- **Configuration:** Database connection strings via Kubernetes Secrets afgehandeld.
- **Service:** Kubernetes Service voor interne load balancing

3.3 Configure a Kubernetes cluster using Ansible

Status: Behaald

Ansible wordt gebruikt voor alle cluster configuratie taken:

- **Cluster Configuration:** Automatische kubeconfig setup en EKS cluster access
- **Application Deployment:** Uitrol van Kubernetes manifests via Ansible templates
- **Secret Management:** Veilige injectie van database credentials
- **Monitoring:** Logfile verzameling van alle applicatie pods

Requirements Check

REQ-15: AWS deployment using Terraform

Alle AWS resources worden volledig via Terraform gedeployed. De infrastructuur is volledig declaratief gedefinieerd. De files zijn te vinden binnen opdracht3/terraform.

REQ-16: Application deployment using IaC

De volledige applicatie deployment gebeurt via Infrastructure as Code met Terraform. De files van terraform zijn te vinden binnen opdracht3/terraform.

REQ-17: Single external IP address

AWS LoadBalancer Service zorgt voor één extern toegangspunt. Zodra de applicatie is uitgerold d.m.v. deploy.sh , zie je in de terminal een link naar de load balancer en via daar alleen zijn de webapplicaties te bereiken.

REQ-18: Docker images on ECR

CloudShirt Docker images worden gebouwd en opgeslagen in AWS ECR repository. Automatische build en push pipeline heb ik geïmplementeerd. Zie hiervoor /opdracht3/terraform/docker-build&push.sh

REQ-19: Automated Kubernetes cluster deployment

EKS cluster wordt volledig geautomatiseerd via Terraform gedeployed, samen met worker nodes en networking.

REQ-20: Master/Slave architecture with 5 replicas

EKS master node beheert worker nodes met 5 CloudShirt applicatie replica's voor load distributie. Zie hiervoor /opdracht3/terraform/3_eks_cluster.tf

REQ-21: Ansible cluster configuration

Ansible playbook configure_cluster.yml configureert het volledige cluster en deployed de applicatie.

REQ-22: Ansible log collection

Ansible playbook collect_logs.yml verzamelt logs van alle applicatie pods naar lokale bestanden.

Belangrijkste Technische Keuzes

Architectuur Beslissingen

- 1. EKS vs Self Managed:** Gekozen voor AWS EKS voor managed control plane, wat de operationele overhead reduceert.
- 2. Terraform Design:** Infrastructuur opgesplitst in logische modules (fundamentals, storage, compute) voor betere onderhoudbaarheid en herbruikbaarheid.
- 3. Ansible Templates:** Kubernetes manifests als Jinja2 templates voor dynamische configuratie injection (ECR URLs, RDS endpoints).
- 4. LoadBalancer Service:** AWS LoadBalancer type voor externe toegang, wat automatisch AWS Application Load Balancer provisioneert.

Deployment Workflow

Uitrol Procedure

Om geheel Opdracht 3 te kunnen uitrollen moet je geheel opdracht3 in de cloudshell omgeving uploaden. Als je vervolgens de deploy.sh exe rechten geeft, kun je hem uitvoeren en wordt alles voor je automatisch uitgerold. Het duurt circa 30-40 min totdat alles stabiel werkt. Aan het eind geeft het script de link naar de loadbalancer, door deze link op je browser in te voeren krijg je toegang tot het systeem.

Schoonmaak Procedure

Om geheel Opdracht 3 te kunnen afbreken moet je geheel opdracht3 in de cloudshell omgeving uploaden. Als je vervolgens de destory.sh exe rechten geeft, kun je hem uitvoeren en wordt alles voor je automatisch afgebroken. Het duurt circa 30 min totdat alles afgebroken is.